Họ và tên: Lương Cao Pháp - 24521288

ASSIGNMENT 6

Chapter 8 (8.8, 8.9, 8.10, 8.11)

Chapter 9 (9.1, 9.2, 9.6, 9.15, 9.16)

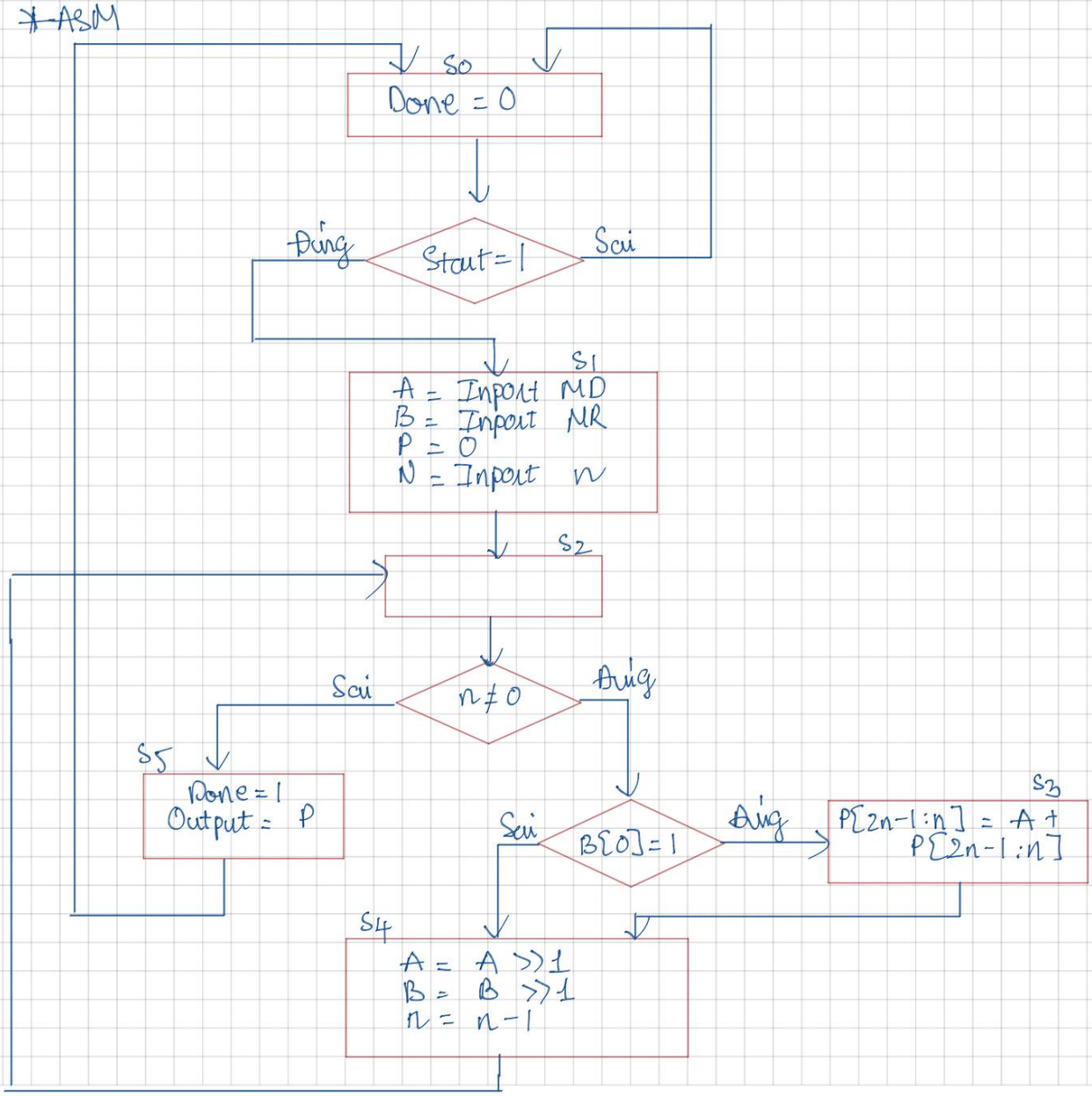
8.8 **(ASM charts) Develop an ASM chart for a shift-and-add multiplier that will look at the least-significant bit of the multiplier (MR), add the multiplicand (MD) to the partial product (PP) if the bit is 1, and then shift the partial product and the multiplier one bit position to the right.**

8.8

* Nhân 2 số n bit không dấu

```
A = Inport MD
B = Inport MR
P = 0
N = Inport n
while (n≠0) {
    if (B[0] ==1) then P[2n-1 : n] ← P[2n-1 :n] + A
    A ← A >> 1
    B ← B >> 1
    n ← n - 1
}
Outport = P
```

* ASM



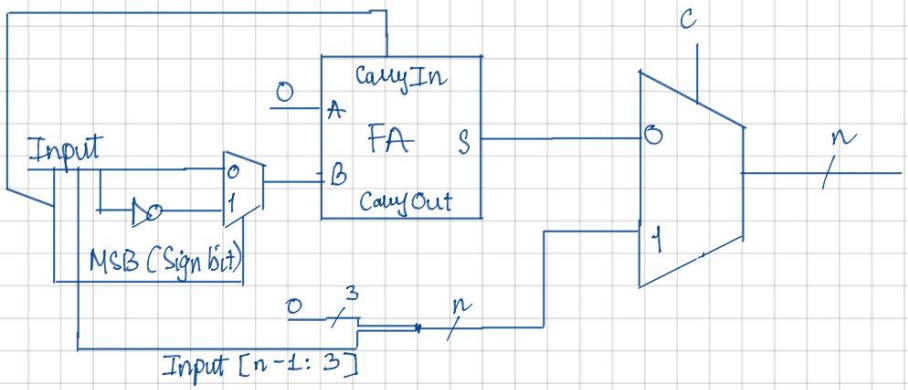
8.9 (Library components) Design units that can perform the following operations.

- **(a)** Absolute value and shift right by three positions
- **(b)** Absolute value and shift right by one and three positions
- **(c)** Add and shift right by three positions
- **(d)** Add and shift right by one and three positions

8.9

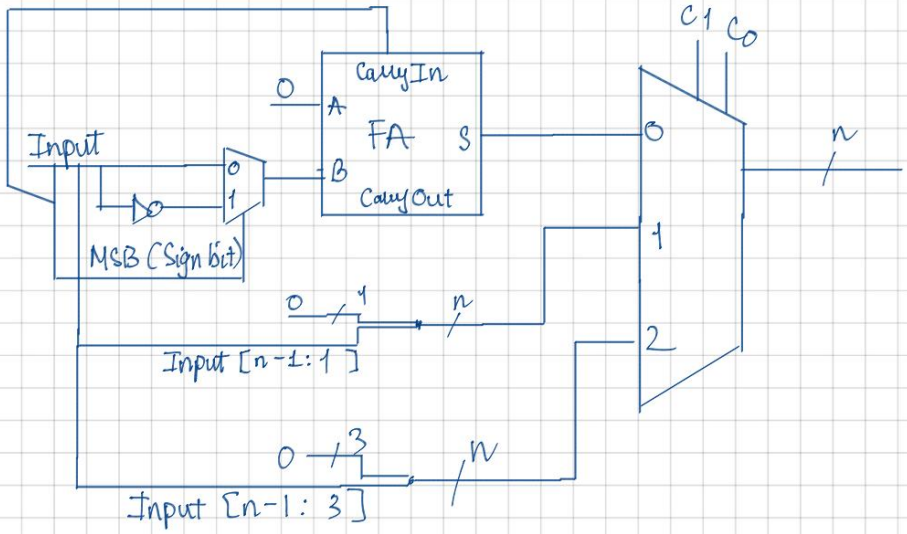
a) [abs / >> 3]

c	Operation
0	Abs
1	Shift Right 3



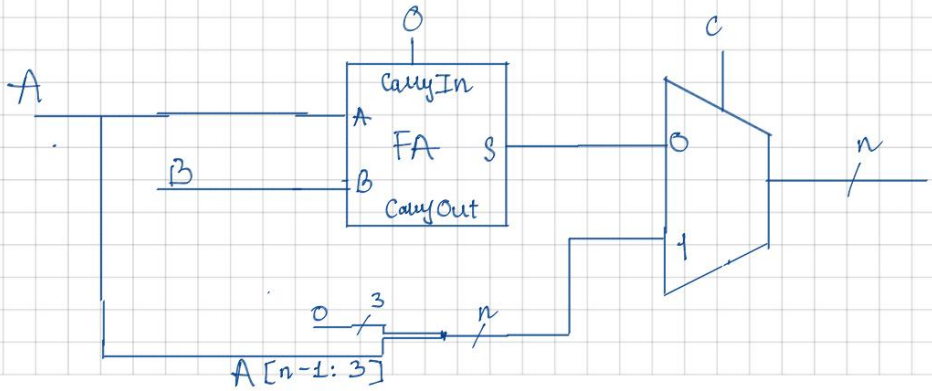
b) [abs / >> 1 / >> 3]

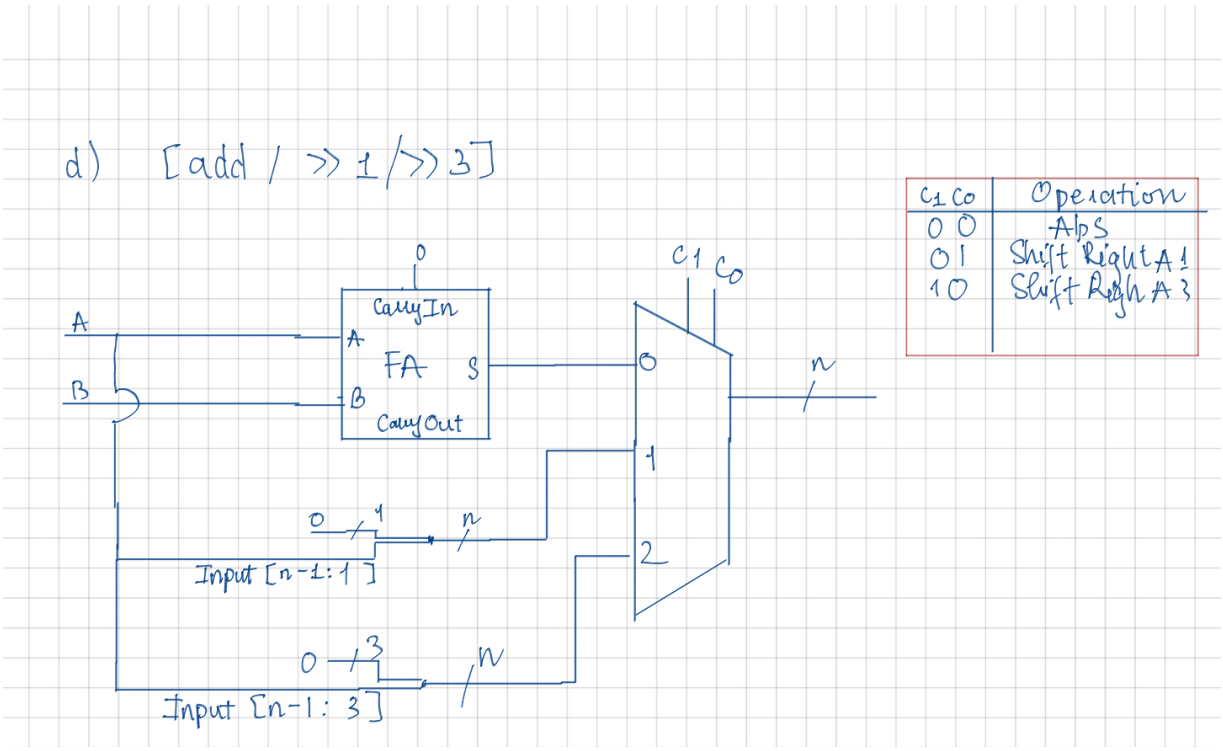
C_1 C_0	Operation
0 0	Abs
0 1	Shift Right 1
1 0	Shift Right 3



c)

c	Operation
0	Add A + B
1	Shift Right A 1 bit





8.10 (Left-edge algorithm) Assign the variables t_1 through t_8 to the minimum number of registers using the left-edge algorithm. The variable lifetimes are given in Figure P8.10.

	s_0	s_1	s_2	s_3	s_4	s_5	s_6	s_7
t_1	×	×	×	×				
t_2	×	×	×					
t_3		×	×	×	×			
t_4				×	×			
t_5					×	×	×	
t_6					×	×		
t_7						×	×	
t_8							×	×

FIGURE P8.10

8.10

	s_0	s_1	s_2	s_3	s_4	s_5	s_6	s_7
t_1	×	×	×	×				
t_2	×	×	×					
t_3		×	×	×	×			
t_4				×	×			
t_5					×	×	×	
t_6					×	×		
t_7						×	×	
t_8							×	×

FIGURE P8.10

Left-edge algorithm:

* Sorted list:

	s_0	s_1	s_2	s_3	s_4	s_5	s_6	s_7
t_1	X	X	X	X				
t_2	X	X	X					
t_3		X	X	X	X			
t_4				X	X			
t_5					X	X	X	
t_6					X	X		
t_7						X	X	
t_8							X	X

* Register assignments:

$R_1 = [t_1; t_5]$

$R_2 = [t_2; t_4; t_7]$

$R_3 = [t_3; t_8]$

$R_4 = [t_6]$

8.11 (Graph partitioning) Group the nodes specified by the compatibility graphs shown in Figure P8.11. Assume that weight on a priority edge to a supernode is equal to the sum of the weights on the priority edges to each node in the supernode.

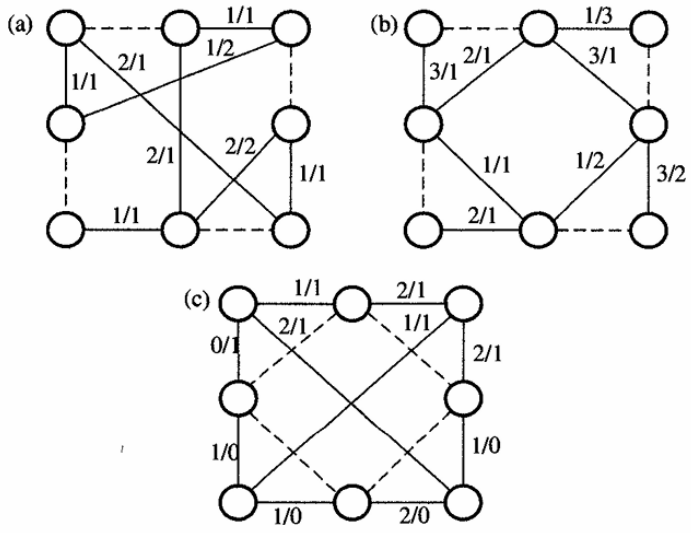


FIGURE P8.11

9.1 (Instruction formats) Write a sequence of instructions that will compute the value of $y = x^2 + 2x + 3$ for a given x using:

(a) Three-address instructions

(b) Two-address instructions

(c) One-address instructions

Assume that no register file is available in the processor except a single accumulator for part (c). Give the number of instructions and memory accesses for each case.

a. Three-address instructions: (no register file)

Mult z, x, x (Mem[z] ← Mem[x] * Mem[x])

Mult y, x, 2 (Mem[y] ← Mem[x] * 2)

Add y, y, z (Mem[y] ← Mem[y] + Mem[z])

Add y, y, 3 (Mem[y] ← Mem[y] + 3)

Giả sử mỗi lệnh chiếm 3 word, chạy trên bộ xử lý 32 bit với mỗi word 32bit.

Số lượng lệnh là: 4

Số lần truy cập bộ nhớ nạp lệnh là: 4 x 3 = 12

Số lần truy cập bộ nhớ lấy toán hạng và lưu kết quả là: 10

b. Two-address instructions:

Move z, x (Mem[z] ← Mem[x])

Mult z, x (Mem[z] ← Mem[z] * Mem[x])

Move y, 3 (Mem[y] ← 3)

Add y, z (Mem[y] ← Mem[y] + Mem[z])

Move z, 2 (Mem[z] ← 2)

Move z, x (Mem[z] ← Mem[x])

Add y, z (Mem[y] ← Mem[y] + Mem[z])

Giả sử mỗi lệnh chiếm 2 word, chạy trên bộ xử lý 32 bit với mỗi word là 32 bit

Số lượng lệnh là 7

Số lần truy cập bộ nhớ nạp lệnh là: 7 x 2 = 14

Số lần truy cập bộ nhớ lấy toán hạng và lưu kết quả là: 15

c. One-address instructions: (A single accumulator register)

Load x (ACC ← Mem[x])

Mult x (ACC ← ACC * Mem[x])
Store z (Mem[z] ← ACC)
Move x (ACC ← Mem[x])
Mult 2 (ACC ← ACC * 2)
Add z (ACC ← ACC + Mem[z])
Add 3 (ACC ← ACC + 3)
Store y (Mem[y] ← ACC)
Giả sử mỗi lệnh chiếm 1 word, chạy trên bộ xử lý 32 bit với mỗi word là 32 bit
Số lượng lệnh là 8
Số lần truy cập bộ nhớ nạp lệnh là: 8 x 1 = 8
Số lần truy cập bộ nhớ lấy toán hạng và lưu kết quả là: 6

9.2 **(Instruction formats) Do Problem 9.1 assuming the availability of a register file. For both register file and memory addresses use:**

(a) One-address instructions
(b) Two-address instructions
(c) Three-address instructions

Minimize the number of memory addresses in each case.

a. One-address instructions
   (Giống như bài 9.1)
b. Two-address instructions
   Load R1, x (RF[1] ← x) #R1 = x
   Move R2, R1 (RF[2] ← RF[1]) # R2 = x
   Mult R2, R2 (RF[2] ← RF[2] * RF[2]) # R2 = x*x
   Add R3, R2 (RF[3] ← RF[2]) # R3 = x*x
   Move R2, R1 (RF[2] ← RF[1]) # R2 = x
   Mult R2, 2 (RF[2] ← RF[2] *2) #R2 = 2x
   Add R3, R2 (RF[3] ← RF[3] + RF[2]) # R3 = x*x + 2x
   Add R3, 3 (RF[3] ← RF[3] +3) # R3 = x*x + 2x + 3
   Store y, R3 (Mem[y] ← RF[3])

c. Three-address instructions

Load R1, x (RF[1] ← x) #R1 = x

Mult R2, R1, R1 (RF[2] ← RF[1] * RF[1]) # R2 = x*x

Mult R3, R1, 2 (RF[2] ← RF[1] *2) #R3 = 2x

Add R3, R3, R2 (RF[3] ← RF[3] + RF[2])

Add R3, R3, 3 (RF[3] ← RF[3] + 3)

Store y, R3 (Mem[y] ← RF[3])

9.6 **(Addressing modes) Write a sequence of instructions that will compute $\sum_{i=1}^{100} a_i x_i$, where $A = [a_1, a_2, \ldots, a_{100}]$ and $X = [x_1, x_2, \ldots, x_{100}]$ represent arrays that are stored in the main memory. Use two-address instructions, in conjunction with:**

(a) Direct addressing

(b) Relative addressing

(c) Indexed addressing with an auto-increment mode

Giả sử địa chỉ mảng A bắt đầu từ địa chỉ là 1000. Địa chỉ mảng X bắt đầu từ địa chỉ là 2000.

a. Direct addressing

Move Sum, 0

Load Temp1, 1000

Load Temp2, 2000

Mult Temp1, Temp2

Add Sum, Temp1

…

Load Temp1, 1099

Load Temp2, 2099

Mult Temp1, Temp2

Add Sum, Temp1

b. Relative Addressing

```
        Move    Sum, 0          # Sum = 0
        Move    IR, 0           #IR = index = 0

Loop:   Loadrel Temp1, IR       # Temp1 = A[IR]
        Loadrel Temp2, IR+100   #Temp2 = X[IR]
        Mult    Temp1, Temp2    #Temp1 = Temp1 * Temp2
        Add     Sum, Temp1      # Sum += Temp1

        Add     IR, 1           #IR = IR + 1
        Cmp     IR, 100
        Bne     Loop
```

c. Indexed addressing with an auto-increment mode

```
Move Sum, 0
Move IR, 0
Loop: Loadindex Temp1, 1000  #Temp1 = A[IR], IR = IR + 1
      Loadindex Tenp2, 2000 # Temp 2 = X[IR], IR = IR + 1
      Mult Temp1, Temp2      # Temp1 = Temp1 * Temp2
      Add Sum, Temp1             # Sum += Temp1

      Cmp IR, 100
      Bne Loop
```

9.15 (IS flowchart) Develop an IS flowchart for the reduced instruction set presented in Figure 9.11.

9.16 (ASM chart) Develop an ASM chart for the reduced instruction set presented in Figure 9.11.